

# DAA – TOPIC 10

## 1. 0/1 Knapsack Problem (Dynamic Programming)

• **Aim:** To solve the 0/1 Knapsack Problem using dynamic programming and find the maximum possible profit within a given capacity.

• **Algorithm:**

1. Create a DP table with rows for items and columns for capacities.
2. For each item, check whether its weight fits the current capacity.
3. If it fits, choose the maximum of including or excluding the item.
4. Otherwise carry forward the previous value.
5. Return the value in the final cell.

### PROGRAM

Python Program

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)]
           for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] +
                    dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
print("Maximum Profit:", knapsack(weights, values, capacity))
```

### OUTPUT

Output

Maximum Profit: 9

• **Conclusion:** The dynamic programming table gives the maximum profit as 9.

## 2. Travelling Salesperson Problem (Dynamic Programming)

• **Aim:** To find the minimum-cost tour that visits every city exactly once and returns to the starting city using dynamic programming.

• **Algorithm:**

1. Represent the distances using a cost matrix.
2. Use a bit mask to represent visited cities.
3. Recursively compute the minimum remaining tour cost.
4. Memoize repeated states.
5. Return to the starting city after all cities are visited.

### PROGRAM

```
Python Program

from functools import lru_cache

def tsp(graph):
    n = len(graph)

    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]

        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(ans,
                           graph[pos][city] +
                           visit(mask | (1 << city), city))

        return ans

    return visit(1, 0)

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print("Minimum Cost:", tsp(graph))
```

### OUTPUT

```
Output

Minimum Cost: 80
```

• **Conclusion:** The DP-based TSP solution finds the minimum tour cost as 80.

### 3. Optimal Binary Search Tree (OBST)

• **Aim:** To construct the optimal cost of a Binary Search Tree from given keys and access frequencies using dynamic programming.

• **Algorithm:**

1. Initialize the cost of single-key trees with their frequencies.
2. Consider chains of increasing length.
3. For each possible root, calculate left and right subtree costs.
4. Add the total frequency of the interval.
5. Store the minimum cost for every interval.

#### PROGRAM

Python Program

```
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total_freq = sum(freq[i:j + 1])

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(cost[i][j],
                                left + right + total_freq)

    return cost[0][n - 1]

keys = [10, 20, 30]
freq = [34, 8, 50]
print("Optimal BST Cost:", optimal_bst(keys, freq))
```

#### OUTPUT

Output

Optimal BST Cost: 142

• **Conclusion:** Dynamic programming gives an optimal BST cost of 142 for the given frequencies.

## 4. 0/1 Knapsack Problem

- **Aim:** To solve the 0/1 Knapsack Problem using a tabulation-based dynamic programming approach.

- **Algorithm:**

1. Initialize a table of zero values.
2. Process each item one by one.
3. For each capacity, compare including and excluding the item.
4. Store the larger value.
5. Return the maximum value for the full capacity.

### PROGRAM

Python Program

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if wt[i - 1] <= w:
                dp[i][w] = max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
print("Maximum Profit =", knapsack(wt, val, W))
```

### OUTPUT

Output

Maximum Profit = 9

- **Conclusion:** The tabulation method produces the maximum profit of 9.

## 5. 0/1 Knapsack Using Memoization

- **Aim:** To solve the 0/1 Knapsack Problem using recursive dynamic programming with memoization.

- **Algorithm:**

1. Check the base case when no items or capacity remain.
2. If the state is already stored, return it.
3. If the item fits, compare taking and skipping it.
4. Otherwise skip the item.
5. Store and return the computed value.

### PROGRAM

Python Program

```
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0
    if memo[n][W] != -1:
        return memo[n][W]

    if wt[n - 1] <= W:
        memo[n][W] = max(
            val[n - 1] +
            knapsack(W - wt[n - 1], wt, val, n - 1, memo),
            knapsack(W, wt, val, n - 1, memo))
    else:
        memo[n][W] = knapsack(W, wt, val, n - 1, memo)
    return memo[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
memo = [[-1] * (W + 1) for _ in range(len(wt) + 1)]
print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))
```

### OUTPUT

Output

Maximum Profit = 9

- **Conclusion:** Memoization avoids repeated subproblems and produces the maximum profit of 9.

## 6. Subset Sum Problem

• **Aim:** To determine whether a subset of the given array adds up exactly to a target sum using dynamic programming.

• **Algorithm:**

1. Create a Boolean DP table.
2. Set the zero-sum state to True for every number of items.
3. For each element, decide whether to include or exclude it.
4. Store the resulting True/False state.
5. Return the target state.

### PROGRAM

Python Program

```
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False] * (target + 1)
          for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for j in range(1, target + 1):
            if arr[i - 1] <= j:
                dp[i][j] = (
                    dp[i - 1][j] or
                    dp[i - 1][j - arr[i - 1]])
            else:
                dp[i][j] = dp[i - 1][j]
    return dp[n][target]

print(subset_sum([3, 34, 4, 12, 5, 2], 9))
```

### OUTPUT

Output

True

• **Conclusion:** The program confirms that a subset with sum 9 exists.

## 7. Travelling Salesperson Problem (DP Version)

• **Aim:** To solve the Travelling Salesperson Problem using memoized dynamic programming and find the minimum tour cost.

• **Algorithm:**

1. Use a graph matrix for travel costs.
2. Represent the set of visited cities with a bit mask.
3. Use memoization for each (mask, position) state.
4. Try every unvisited city.
5. Return to city 0 after all cities are visited.

### PROGRAM

```
Python Program

from functools import lru_cache

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)

@lru_cache(None)
def tsp(mask, pos):
    if mask == (1 << n) - 1:
        return graph[pos][0]

    ans = float('inf')
    for city in range(n):
        if not mask & (1 << city):
            ans = min(ans,
                      graph[pos][city] +
                      tsp(mask | (1 << city), city))

    return ans

print("Minimum Cost =", tsp(1, 0))
```

### OUTPUT

```
Output

Minimum Cost = 80
```

• **Conclusion:** The memoized DP version finds the minimum travelling cost as 80.

## 8. Travelling Salesperson Using DP Table

- **Aim:** To solve TSP using an iterative dynamic programming table instead of recursion.

- **Algorithm:**

1. Create a DP table indexed by visited-city masks and current city.
2. Set the starting state to zero cost.
3. For each state, try every unvisited destination.
4. Update the destination state with the smaller cost.
5. Add the return edge to the starting city.

### PROGRAM

Python Program

```
def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0

    for mask in range(1 << n):
        for u in range(n):
            if mask & (1 << u):
                for v in range(n):
                    if not mask & (1 << v):
                        dp[mask | (1 << v)][v] = min(
                            dp[mask | (1 << v)][v],
                            dp[mask][u] + graph[u][v])

    ans = float('inf')
    for i in range(1, n):
        ans = min(ans,
                  dp[(1 << n) - 1][i] + graph[i][0])

    return ans

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print("Minimum Cost =", tsp_dp(graph))
```

### OUTPUT

Output

Minimum Cost = 80

- **Conclusion:** The table-based dynamic programming approach also produces a minimum cost of 80.



## 9. Optimal Binary Search Tree (OBST)

- **Aim:** To calculate the minimum search cost of an Optimal Binary Search Tree from key frequencies.

- **Algorithm:**

1. Initialize single-key costs.
2. Build costs for intervals of increasing length.
3. Try each key as the root.
4. Combine left cost, right cost, and interval frequency.
5. Store the minimum interval cost.

### PROGRAM

```
Python Program

def optimal_bst(freq):
    n = len(freq)
    cost = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for L in range(2, n + 1):
        for i in range(n - L + 1):
            j = i + L - 1
            cost[i][j] = float('inf')
            total = sum(freq[i:j + 1])

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(
                    cost[i][j],
                    left + right + total)
            return cost[0][n - 1]

print("Optimal Cost =", optimal_bst([34, 8, 50]))
```

### OUTPUT

```
Output
Optimal Cost = 142
```

- **Conclusion:** The minimum search cost for the supplied frequencies is 142.

## 10. Matrix Chain Multiplication

• **Aim:** To find the minimum number of scalar multiplications needed to multiply a chain of matrices using dynamic programming.

• **Algorithm:**

1. Create a DP table for matrix-chain intervals.
2. Consider chains of increasing length.
3. Try every possible split point.
4. Calculate the multiplication cost for each split.
5. Store the minimum cost for each interval.

### PROGRAM

Python Program

```
def matrix_chain(arr):
    n = len(arr)
    dp = [[0] * n for _ in range(n)]

    for L in range(2, n):
        for i in range(1, n - L + 1):
            j = i + L - 1
            dp[i][j] = float('inf')

            for k in range(i, j):
                q = (dp[i][k] +
                    dp[k + 1][j] +
                    arr[i - 1] * arr[k] * arr[j])
                dp[i][j] = min(dp[i][j], q)
    return dp[1][n - 1]

print("Minimum Multiplications =",
      matrix_chain([10, 20, 30, 40, 30]))
```

### OUTPUT

Output

Minimum Multiplications = 30000

• **Conclusion:** The optimal parenthesization requires 30000 scalar multiplications.

## 11. Coin Change (Minimum Coins)

- **Aim:** To find the minimum number of coins needed to make a given amount using dynamic programming.

- **Algorithm:**

1. Initialize  $dp[0] = 0$  and the remaining states to infinity.
2. For each amount, try every available coin.
3. Update the state using one additional coin.
4. Continue until the target amount is reached.
5. Return the minimum coin count.

### PROGRAM

```
Python Program

def min_coins(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i - c] + 1)
    return dp[amount]

print("Minimum Coins =", min_coins([1, 2, 5], 11))
```

### OUTPUT

```
Output

Minimum Coins = 3
```

- **Conclusion:** The minimum number of coins required to make 11 is 3.

## 12. Rod Cutting Problem

- **Aim:** To maximize revenue by cutting a rod into pieces using dynamic programming.

- **Algorithm:**

1. Create a DP array where  $dp[i]$  is the best revenue for length  $i$ .
2. For each length, try every possible first cut.
3. Combine the price of the cut with the best revenue for the remaining length.
4. Store the maximum revenue.
5. Return the value for the full rod length.

### PROGRAM

Python Program

```
def rod_cut(price, n):
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        for j in range(i):
            dp[i] = max(
                dp[i],
                price[j] + dp[i - j - 1])
    return dp[n]

print("Maximum Revenue =",
      rod_cut([1, 5, 8, 9, 10, 17, 17, 20], 8))
```

### OUTPUT

Output

Maximum Revenue = 22

- **Conclusion:** The optimal cutting strategy gives a maximum revenue of 22.

### 13. Minimum Cost Path

• **Aim:** To find the minimum cost of travelling from the top-left cell to the bottom-right cell of a cost matrix using dynamic programming.

• **Algorithm:**

1. Initialize the first cell.
2. Build the first row and first column using cumulative costs.
3. For each remaining cell, choose the smaller cost from the top or left.
4. Add the current cell cost.
5. Return the bottom-right DP value.

**PROGRAM**

Python Program

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0] * n for _ in range(m)]

    dp[0][0] = cost[0][0]

    for i in range(1, m):
        dp[i][0] = dp[i - 1][0] + cost[i][0]

    for j in range(1, n):
        dp[0][j] = dp[0][j - 1] + cost[0][j]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = cost[i][j] + min(
                dp[i - 1][j], dp[i][j - 1])
    return dp[m - 1][n - 1]

cost = [
    [1, 3, 1],
    [1, 5, 1],
    [4, 2, 1]
]
print("Minimum Cost =", min_cost(cost))
```

**OUTPUT**

Output

Minimum Cost = 7

• **Conclusion:** The minimum cost path for the supplied matrix has total cost 7.